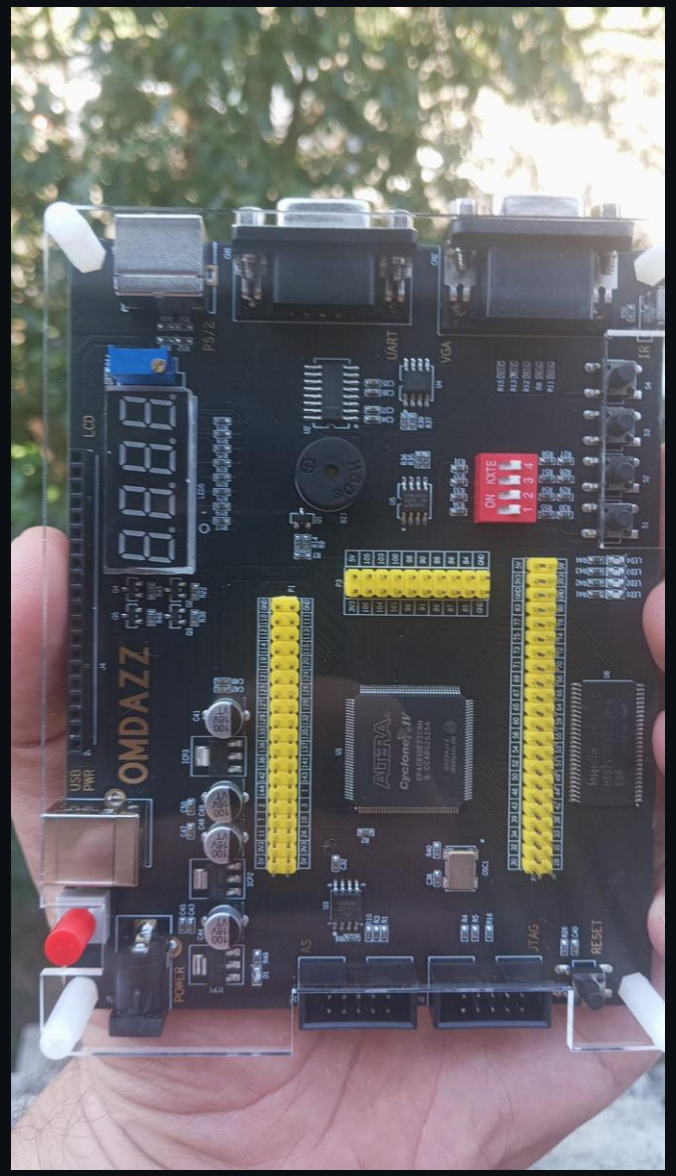
https://www.altera.com/downloads/fpga-development-tools/quartus-prime-lite-edition-design-software-version-25-1-windows

LAB 1 - FPGA

From an unpacked board to the first programmed LED

Level: absolute beginner • OMDAZZ / Altera Cyclone IV EP4CE10E22C8N



Laboratory goal

Today you will power the board, connect the USB-Blaster, create a Quartus project, compile it, configure the FPGA, and turn on an LED using a button.

Practical work: every step is performed on the physical board, not only explained.

Abbrev.: FPGA = Field-Programmable Gate Array; LED = Light-Emitting Diode.

What you will be able to do by the end

No previous experience with FPGA, JTAG, or Quartus is assumed.

- You will identify on the board: power input, the POWER button, JTAG, the FPGA, the push buttons, and the LEDs.
- You will power the board correctly at 5 V and verify the power indicator.
- You will connect the USB-Blaster programmer to the PC and to the JTAG connector on the board.
- You will create a project for EP4CE10E22C8N and a Verilog file.
- You will perform pin assignment, Compile, and Program/Configure.
- You will test it physically: press S1 → LED1 turns on.

Abbrev.: FPGA = Field-Programmable Gate Array; JTAG = Joint Test Action Group; PC = Personal Computer; LED = Light-Emitting Diode.

OMDAZZ board — complete visual map

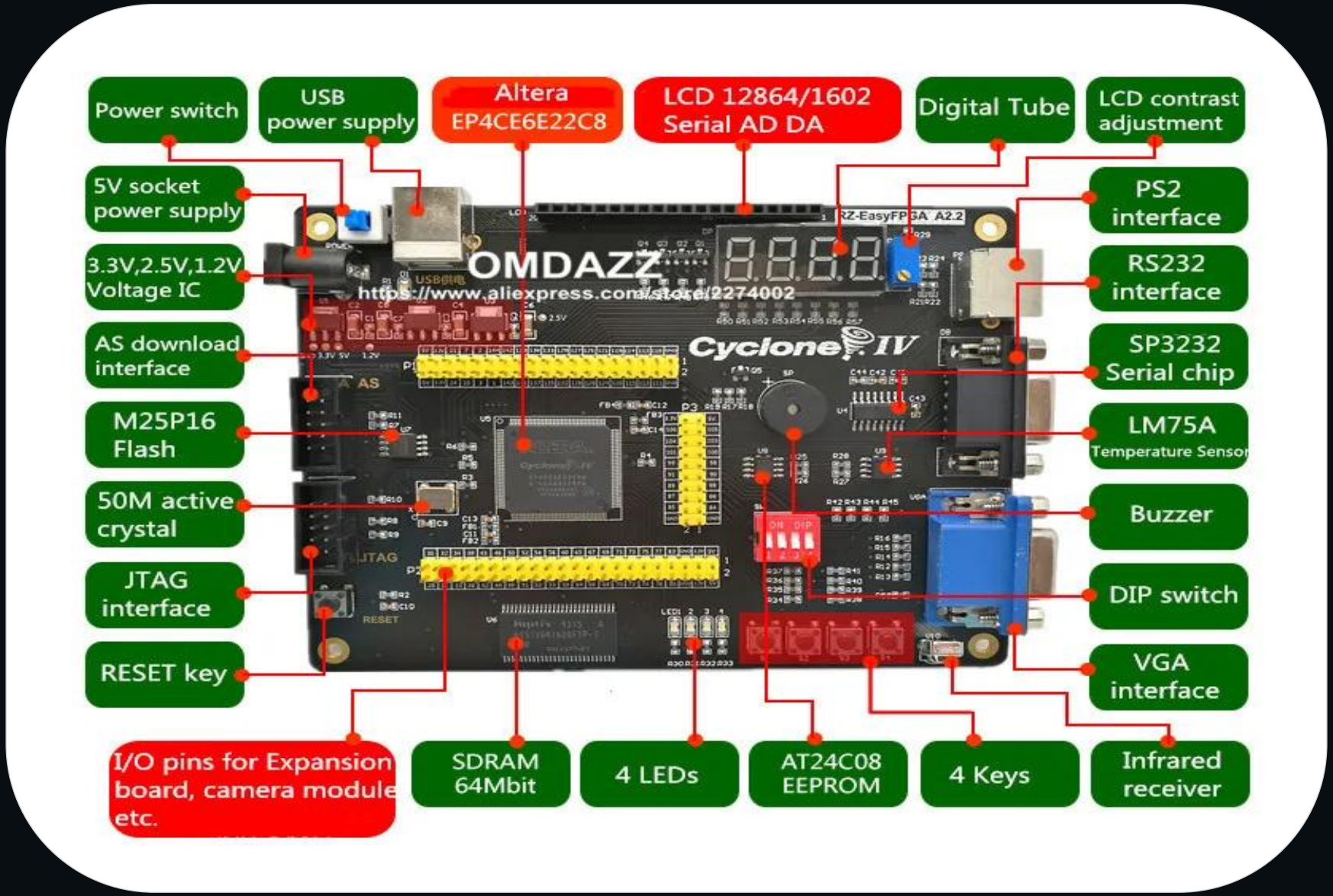
In the first lab you do not use all modules. Identify only power, JTAG, buttons, LEDs, and the FPGA.

Abbrev.:

USB = Universal Serial Bus;
PWR = Power;
JTAG = Joint Test Action Group;
AS = Active Serial;
I/O = Input/Output;
LCD = Liquid Crystal Display;
VGA = Video Graphics Array;
DIP = Dual In-line Package;
SDRAM = Synchronous Dynamic RAM;

EEPROM = Electrically Erasable Programmable ROM;

IR = Infrared.



For Lab 1: USB PWR = power input, USB-Blaster + JTAG = programming, KEY = input, LED = output.

Mandatory abbreviations before connecting anything

These names appear on the board and in Quartus. You must understand them before touching the cables.

FPGA = Field-Programmable Gate Array
Reconfigurable digital circuit. It does not execute a program like a CPU; after synthesis it contains connections, LUTs, and flip-flops.

JTAG = Joint Test Action Group
The interface through which Quartus sends the .SOF file to the FPGA. In this lab, configuration is done through JTAG.

AS = Active Serial
Interface used for configuration from Flash memory. Do not use it in the first lab; do not connect the USB-Blaster here.

USB PWR = USB Power
The square USB connector on the board is for power. It does not replace the USB-Blaster for programming.

PWR / POWER = power
The red POWER button turns the board on/off. The supply must be 5 V.

I/O = Input/Output
Pins through which the FPGA receives signals or drives signals. KEY is an input; LED is an output.

KEY / S1–S4 = push buttons
The buttons on the board produce logic values for the FPGA. In the first test, pressing a button turns on an LED.

LED = Light-Emitting Diode
The LED is the visible output. If configuration succeeded, pressing the button must change the LED state.

Safety rule: the board is powered separately; programming is performed through the USB-Blaster connected to JTAG.

Other board abbreviations — do not use them yet

You should recognize them on the board, but in Lab 1 you do not use these peripherals.

DIP switch = Dual In-line Package switch
Small ON/OFF switches. They can become logic inputs, but we use them only after the KEY test.

LCD = Liquid Crystal Display
Liquid crystal display. The board mentions the 12864/1602 variants.

VGA = Video Graphics Array
Analog video interface. It requires strictly timed signals; it is not used in the first lab.

RS232 / SP3232
RS232 is a serial standard. SP3232 is the level-shifting chip for this communication.

SDRAM = Synchronous Dynamic RAM
External synchronous memory. The board has 64 Mbit SDRAM.

EEPROM = Electrically Erasable Programmable ROM
Non-volatile memory: it keeps data without power. Here it appears as AT24C08.

Flash M25P16
Serial Flash memory used to store configuration or data.

50M active crystal
50 MHz oscillator. It can become the clock signal for sequential circuits.

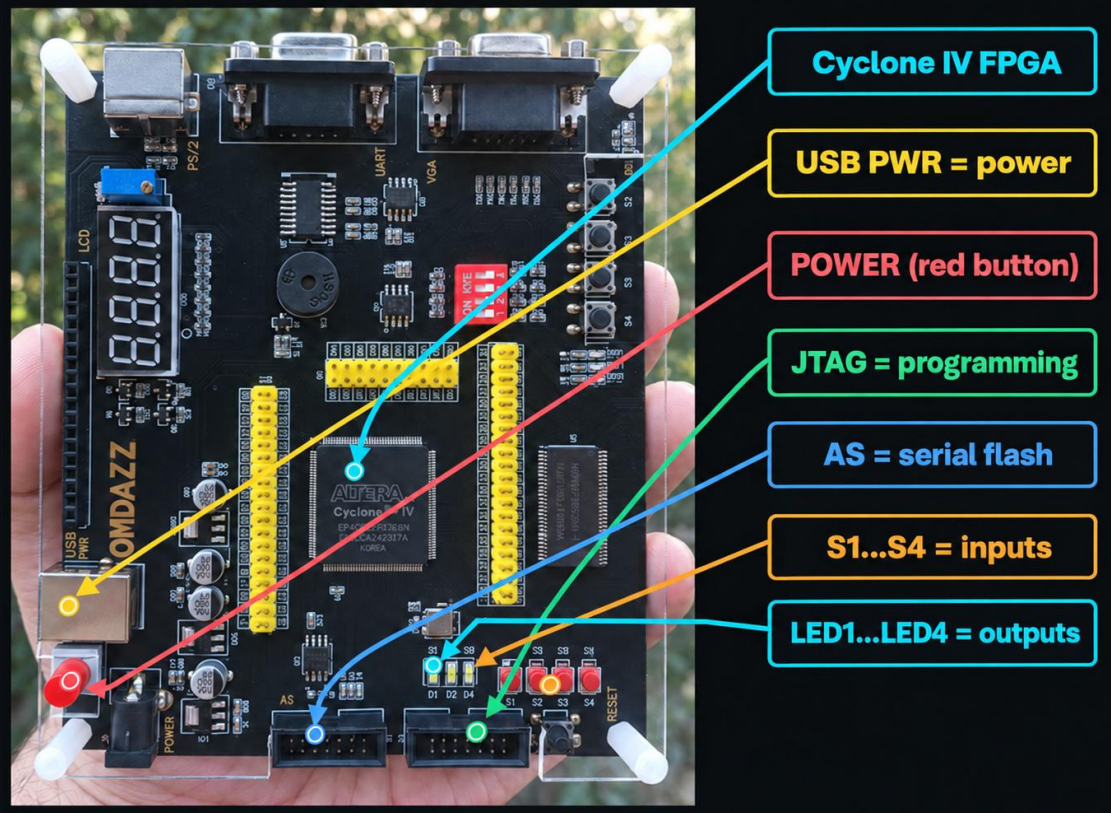
Buzzer / LM75A / IR
Buzzer = sound output; LM75A = temperature sensor; IR = infrared receiver.

SOF / POF
.SOF temporarily configures the FPGA through JTAG. .POF is used to program the Flash memory.

In Lab 1 the objective stays simple: correct power → JTAG detected → .SOF loaded → KEY controls LED.

1. First: get to know the board

Do not connect anything until you know what each important connector is.



Today we use only:

USB PWR + POWER + JTAG + one button + one LED. The rest (VGA, UART, 7-segment display, buzzer, SDRAM) stays for later labs.

Very important

The USB-B connector marked “USB PWR” powers the board. It is not the interface used to download the project into the FPGA.

Programming

Programming is done with an external USB-Blaster programmer connected to the JTAG connector on the board.

Abbrev.: USB = Universal Serial Bus; PWR = Power; JTAG = Joint Test Action Group; VGA = Video Graphics Array; UART = Universal Asynchronous Receiver/Transmitter; SDRAM = Synchronous Dynamic RAM.

2. What every lab bench must have

A. FPGA board

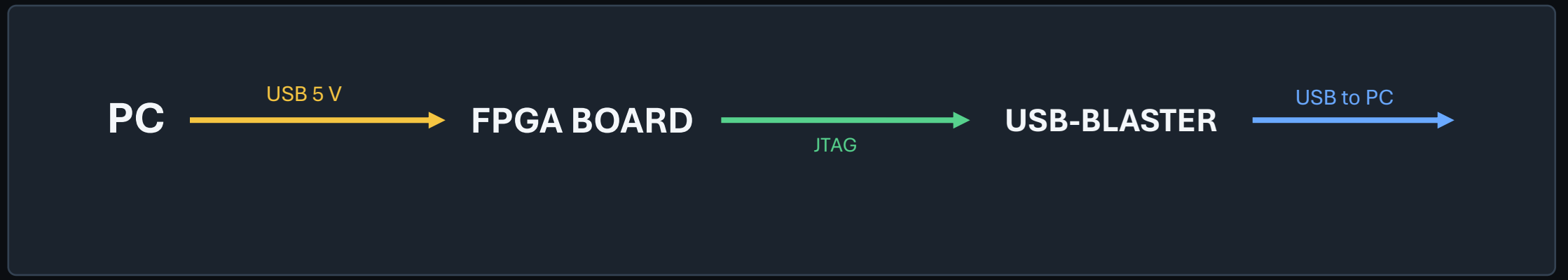
OMDAZZ with Altera Cyclone IV EP4CE10E22C8N.

B. USB A → B cable

It goes into the board connector marked USB PWR. Role: 5 V POWER.

C. USB-Blaster

JTAG programmer + its USB/mini-USB cable + 10-pin ribbon cable.



There are two different USB connections: one powers the board, the other connects the USB-Blaster to the PC.

Abbrev.: FPGA = Field-Programmable Gate Array; USB = Universal Serial Bus; JTAG = Joint Test Action Group; PC = Personal Computer.

3. Connection rule - before powering on

1. **The board is OFF**

   The POWER button is not pressed. Connect cables while the board is off.

2. **Use 5 V**

   This board requires 5 V. Do not use a 9 V or 12 V adapter.

3. **Choose one power source only**

   For this lab: power from USB PWR. Do not power it simultaneously through the DC jack.

4. **Do not force the JTAG connector**

   Align the connector/ribbon correctly using the mechanical key or the pin-1 mark. If it does not fit easily, check the orientation.

5. Connect/disconnect the ribbon cable
   with the board off
   This is the simple and safe procedure for this lab.

Abbrev.: JTAG = Joint Test Action Group; USB PWR = Universal Serial Bus Power; DC = Direct Current; V = volt.

4. POWERING THE BOARD - exactly what you do

1

Plug the USB A → B cable into “USB PWR”

This is the square USB-B connector at the lower-left side of the board.

2

Plug the USB-A end into the PC

The PC supplies the 5 V needed by the board.

3

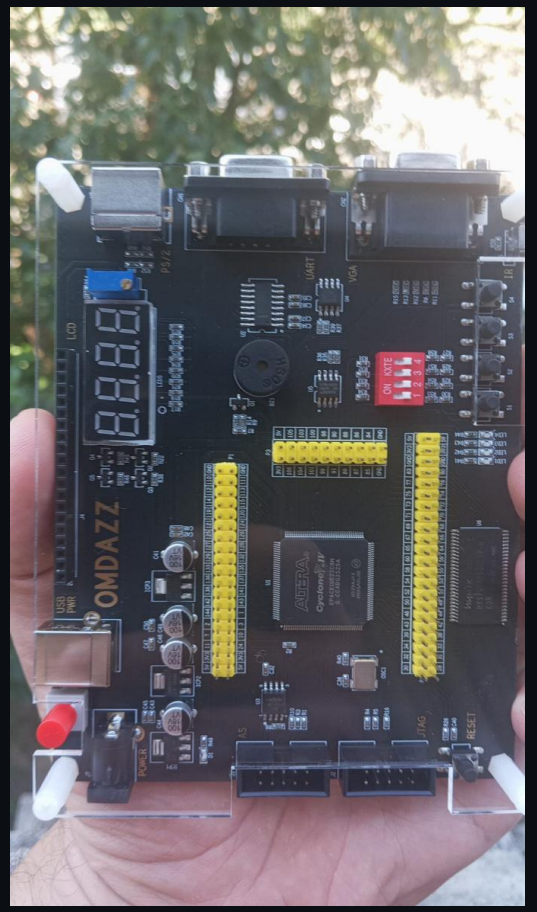
Press the red POWER button

It is a latching button. One press turns the board on; the next press turns it off.

4

Check the power LED

A power indicator light must turn on. If it does not, STOP and check the cable/source.



STOP here if the board has no power. Do not try to program a board that is not powered on.

Abbrev.: USB = Universal Serial Bus; PWR = Power; PC = Personal Computer; LED = Light-Emitting Diode; V = volt.

5. POWER ≠ RESET

Two buttons with completely different roles.

POWER - the red button

It removes or applies power to the board. If you turn it off, the FPGA and all peripherals lose power.

RESET - the small lower button

It resets the circuit/configuration according to the board wiring. It is not the main power button.

For now:

- Turn the board on/off only using POWER.
- Do not use RESET to “turn on” the board.
- After programming, use RESET only if your design explicitly handles it.

Abbrev.: FPGA = Field-Programmable Gate Array; POWER = power switch; RESET = reset signal/button.

6. Why the USB PWR cable does not program the FPGA

USB PWR

5 V → regulatoare → 3.3 V / 2.5 V / 1.2 V

Role: POWER

USB-Blaster + JTAG

PC → USB-Blaster → JTAG → FPGA

Role: PROGRAMMING / CONFIGURATION

If you connect only USB PWR, the board is powered, but Quartus has no path for sending the bitstream.

Abbrev.: USB PWR = Universal Serial Bus Power; FPGA = Field-Programmable Gate Array; JTAG = Joint Test Action Group; PC = Personal Computer; V = volt.

7. Connecting the USB-Blaster to JTAG

1. **Turn the board OFF**

   POWER off before connecting the JTAG ribbon cable.

2. **Connect the USB-Blaster to the PC**

   Use the programmer USB/mini-USB cable.

3. **Connect the 10-pin ribbon cable to JTAG**

   Use the connector marked JTAG, NOT the AS connector.

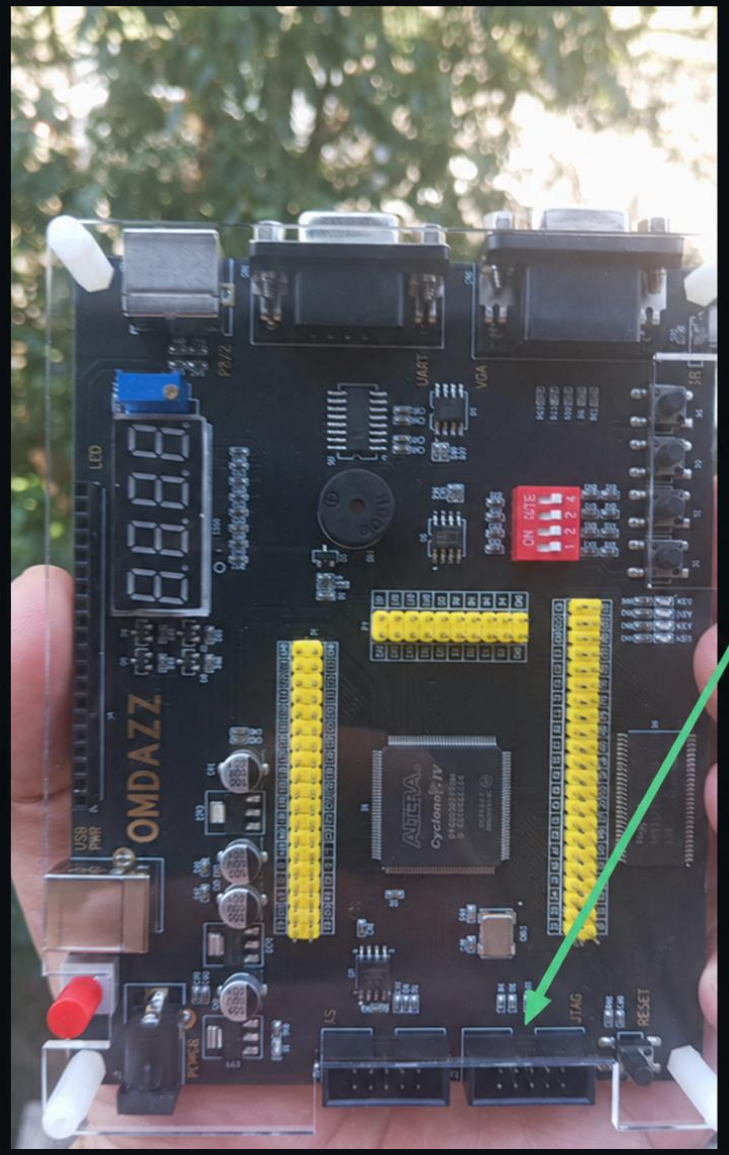
4. **Turn the board back ON**

   USB PWR connected + POWER button pressed.

5. Only now open Quartus Programmer

   The hardware must be powered so the JTAG chain can be detected.



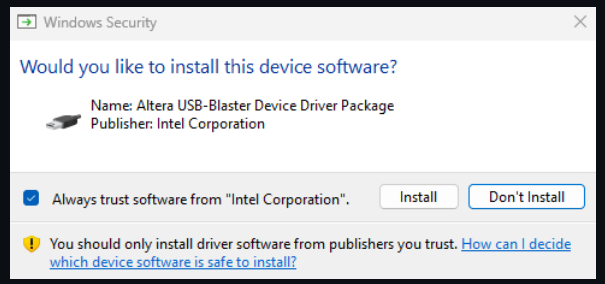


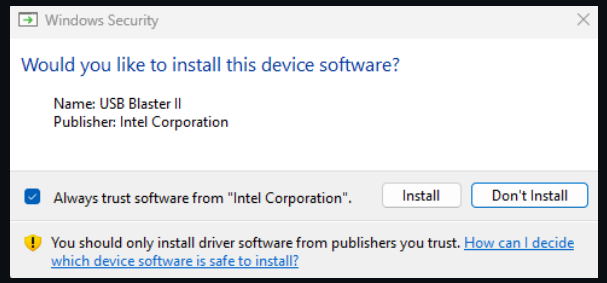
Abbrev.: USB = Universal Serial Bus; JTAG = Joint Test Action Group; AS = Active Serial; PWR = Power.

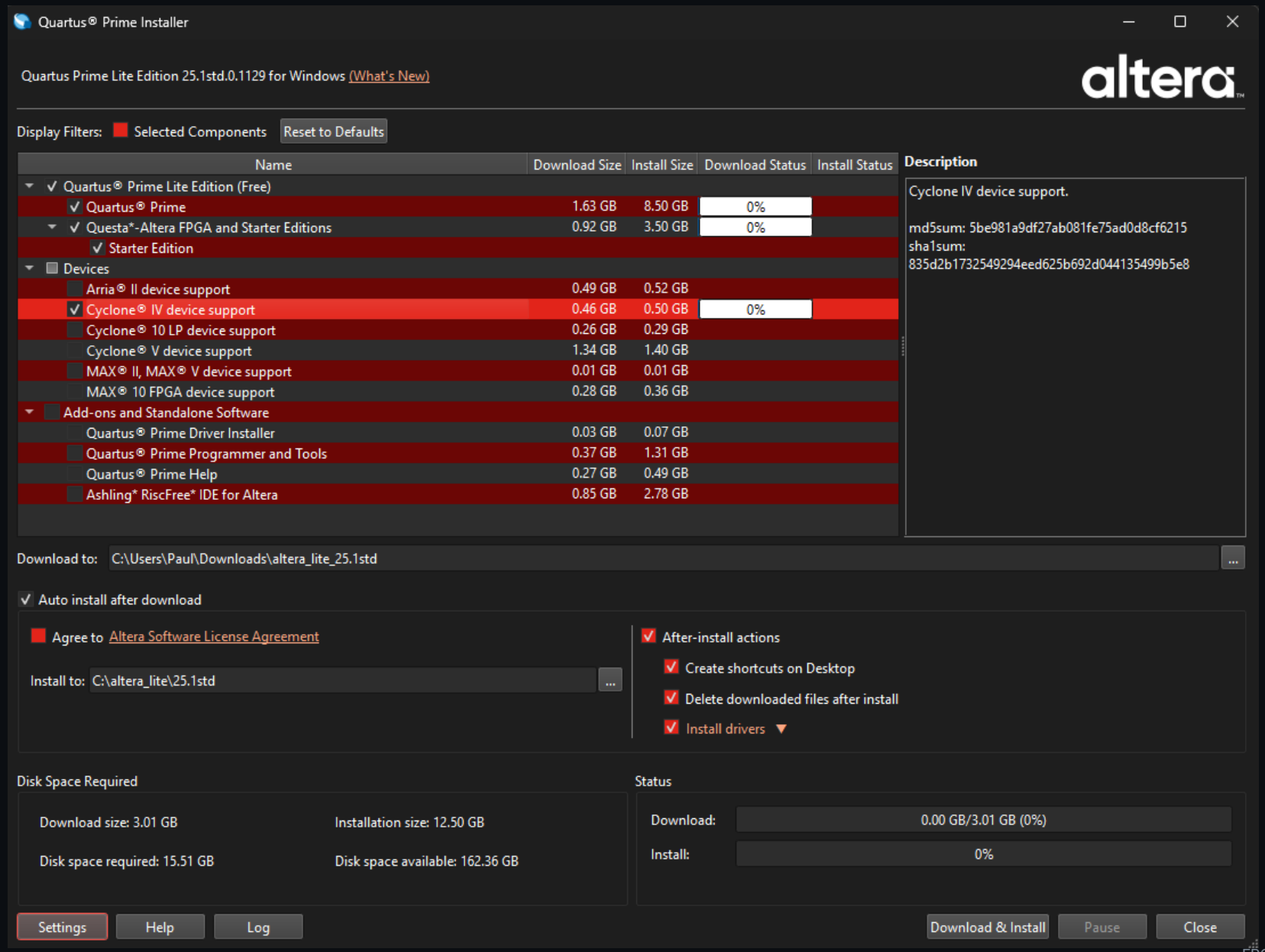
https://www.altera.com/downloads/fpga-development-tools/quartus-prime-lite-edition-design-software-version-25-1-windows

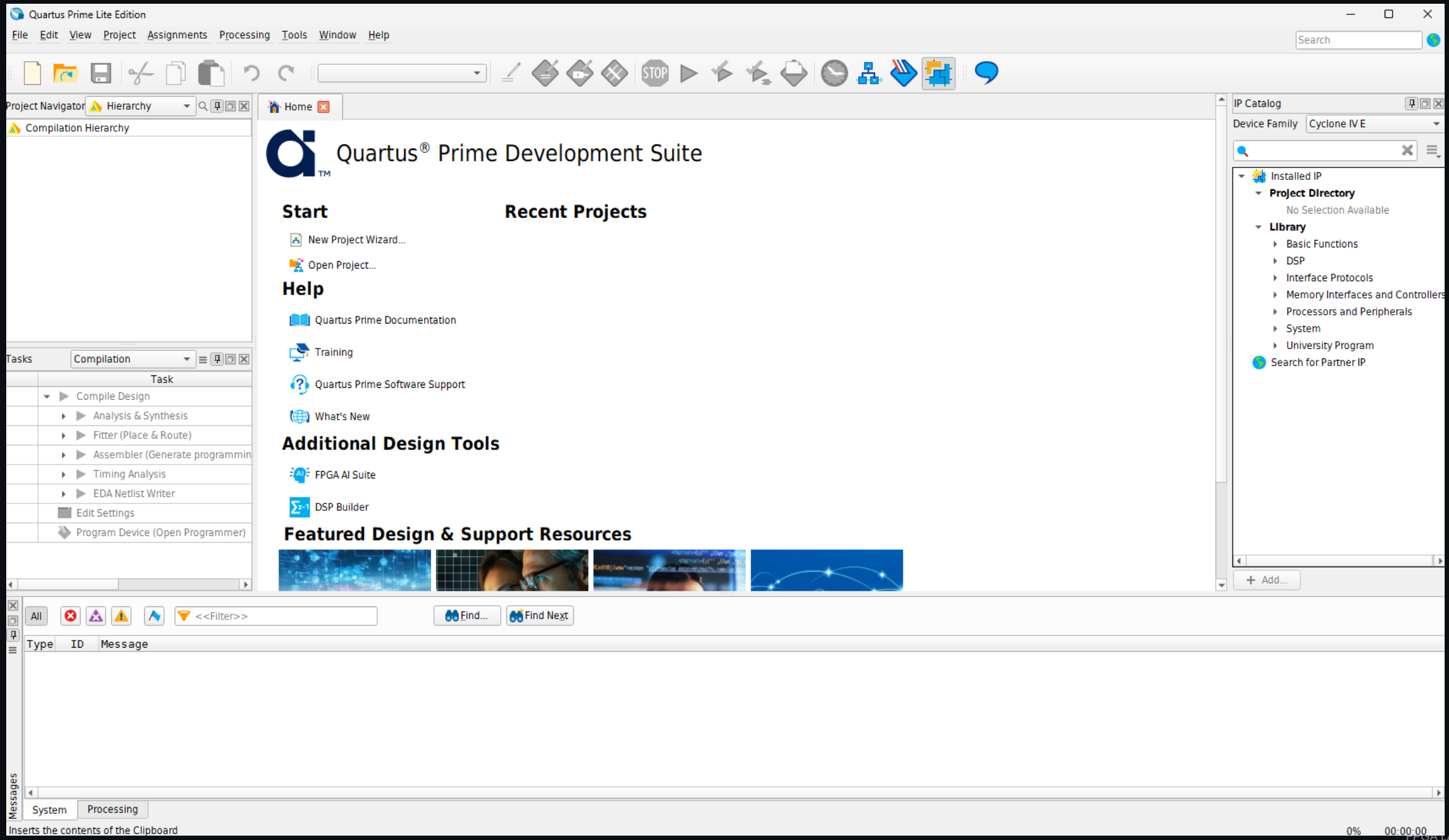
- ✓ Quartus Prime
- ✓ Cyclone IV device support
- ✓ Install drivers

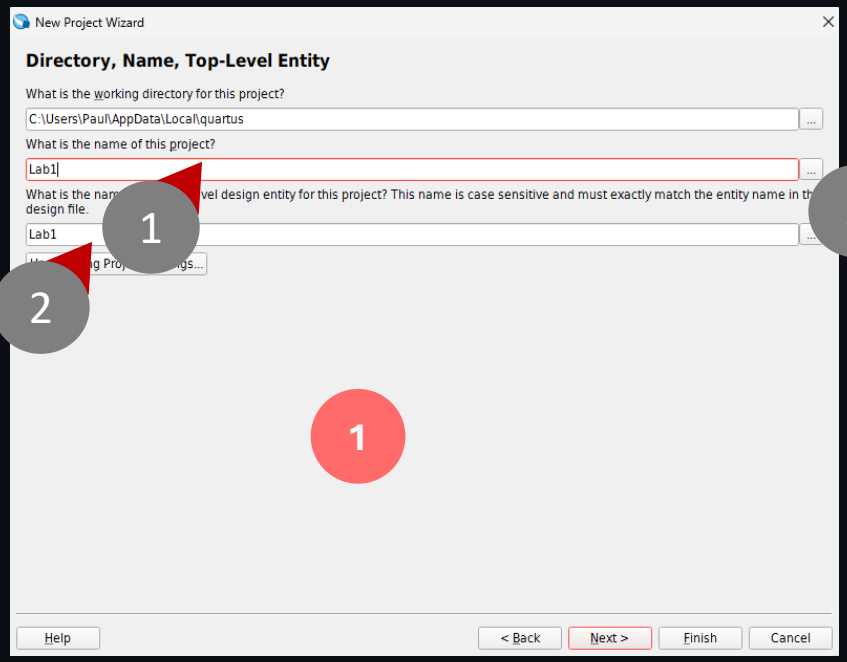
- ☐ Questa Starter Edition

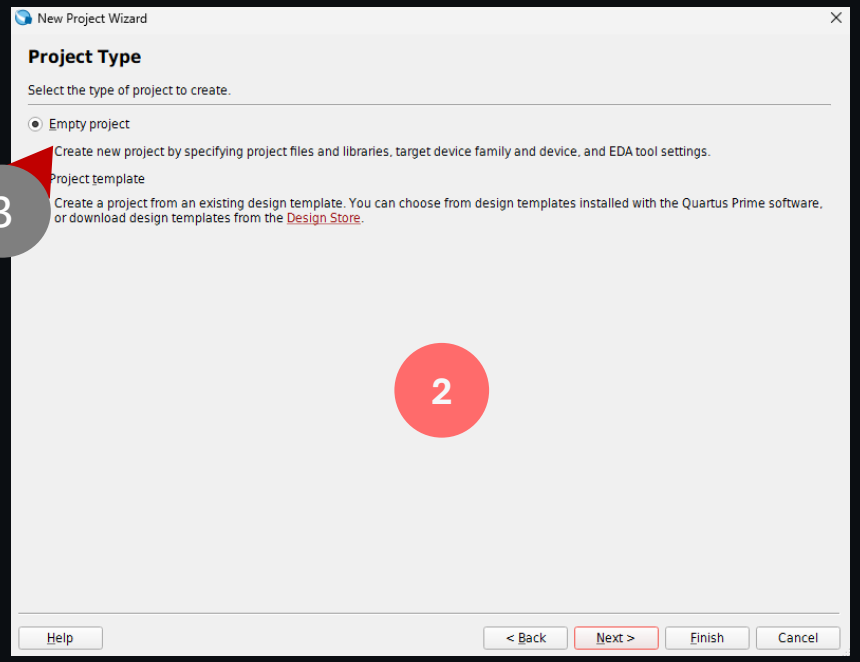


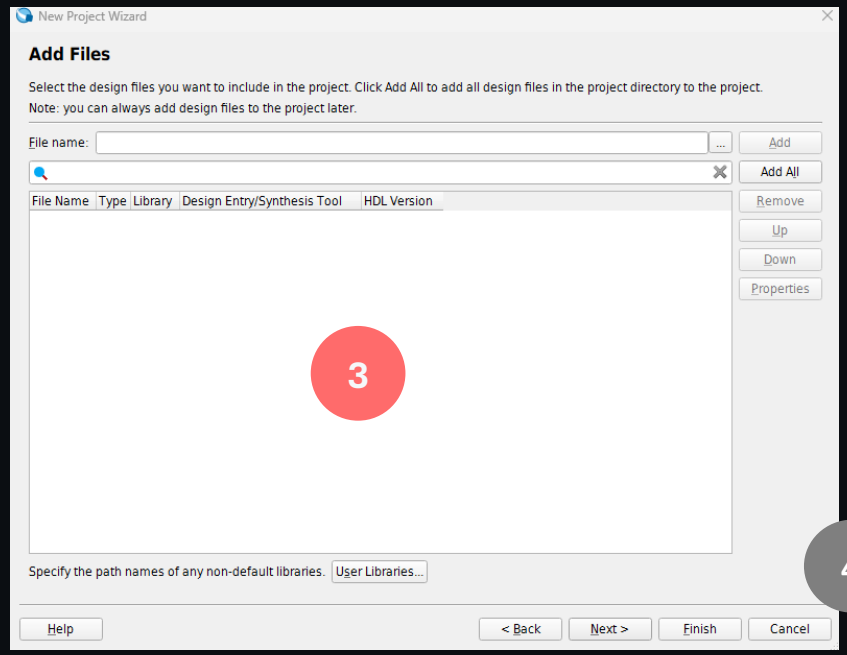


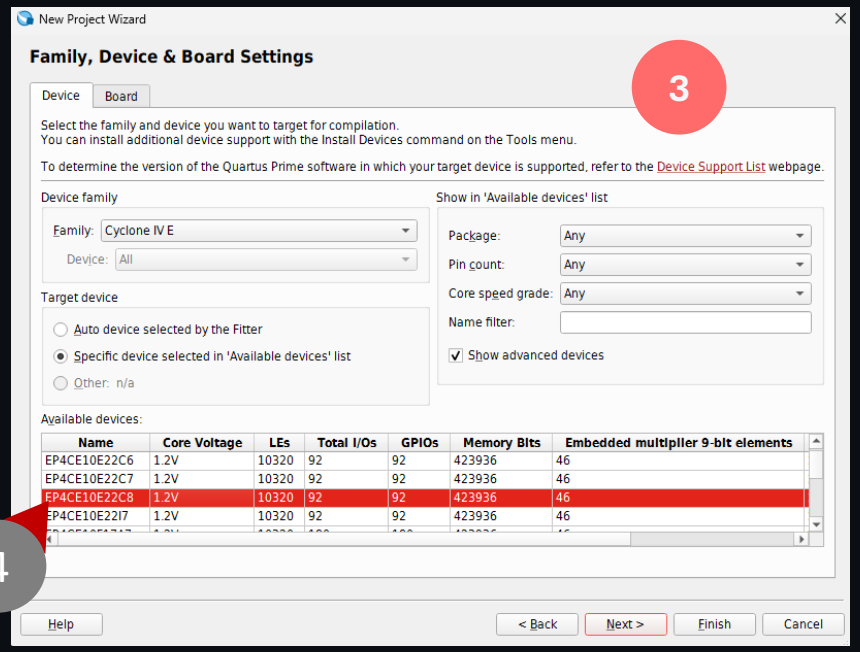


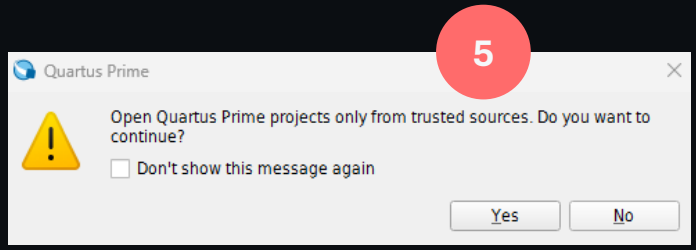

Quartus Prime Lite Edition
File Edit View Project Assignments Processing Tools Window Help
Search
Project Navigator Hierarchy
Compilation Hierarchy
Tasks Compilation
Task
Compile Design
Analysis & Synthesis
Fitter (Place & Route)
Assembler (Generate programmin
Timing Analysis
EDA Netlist Writer
Edit Settings
Program Device (Open Programmer)
Home
Quartus® Prime Development Suite
Start
New Project Wizard...
Open Project...
Recent Projects
Help
Quartus Prime Documentation
Training
Quartus Prime Software Support
What's New
Additional Design Tools
FPGA AI Suite
DSP Builder
Featured Design & Support Resources
IP Catalog
Device Family Cyclone IV E
Installed IP
Project Directory
No Selection Available
Library
Basic Functions
DSP
Interface Protocols
Memory Interfaces and Controllers
Processors and Peripherals
System
University Program
Search for Partner IP
Add...
All
<<Filter>>
Find...
Find Next
Type ID Message
Messages
System
Processing
Inserts the contents of the Clipboard
0%
00:00:00

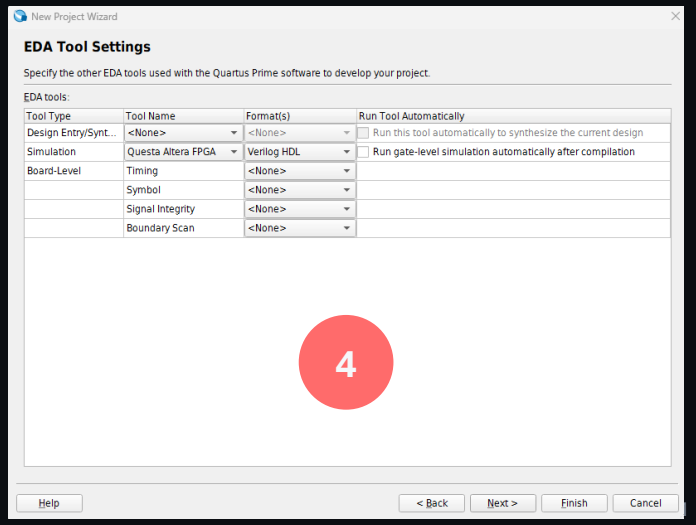

New Project Wizard
Directory, Name, Top-Level Entity
What is the working directory for this project?
C:\Users\Paul\AppData\Local\quartus
What is the name of this project?
Lab1
What is the name of the top-level design entity for this project? This name is case sensitive and must exactly match the entity name in the design file.
Lab1
Help
< Back
Next >
Finish
Cancel
1
2


New Project Wizard
Project Type
Select the type of project to create.
Empty project
Create new project by specifying project files and libraries, target device family and device, and EDA tool settings.
Project template
Create a project from an existing design template. You can choose from design templates installed with the Quartus Prime software, or download design templates from the Design Store.
Help
< Back
Next >
Finish
Cancel
2
3


New Project Wizard
Add Files
Select the design files you want to include in the project. Click Add All to add all design files in the project directory to the project.
Note: you can always add design files to the project later.
File name:
Add
Add All
Remove
Up
Down
Properties
File Name
Type
Library
Design Entry/Synthesis Tool
HDL Version
Specify the path names of any non-default libraries.
User Libraries...
Help
< Back
Next >
Finish
Cancel
3

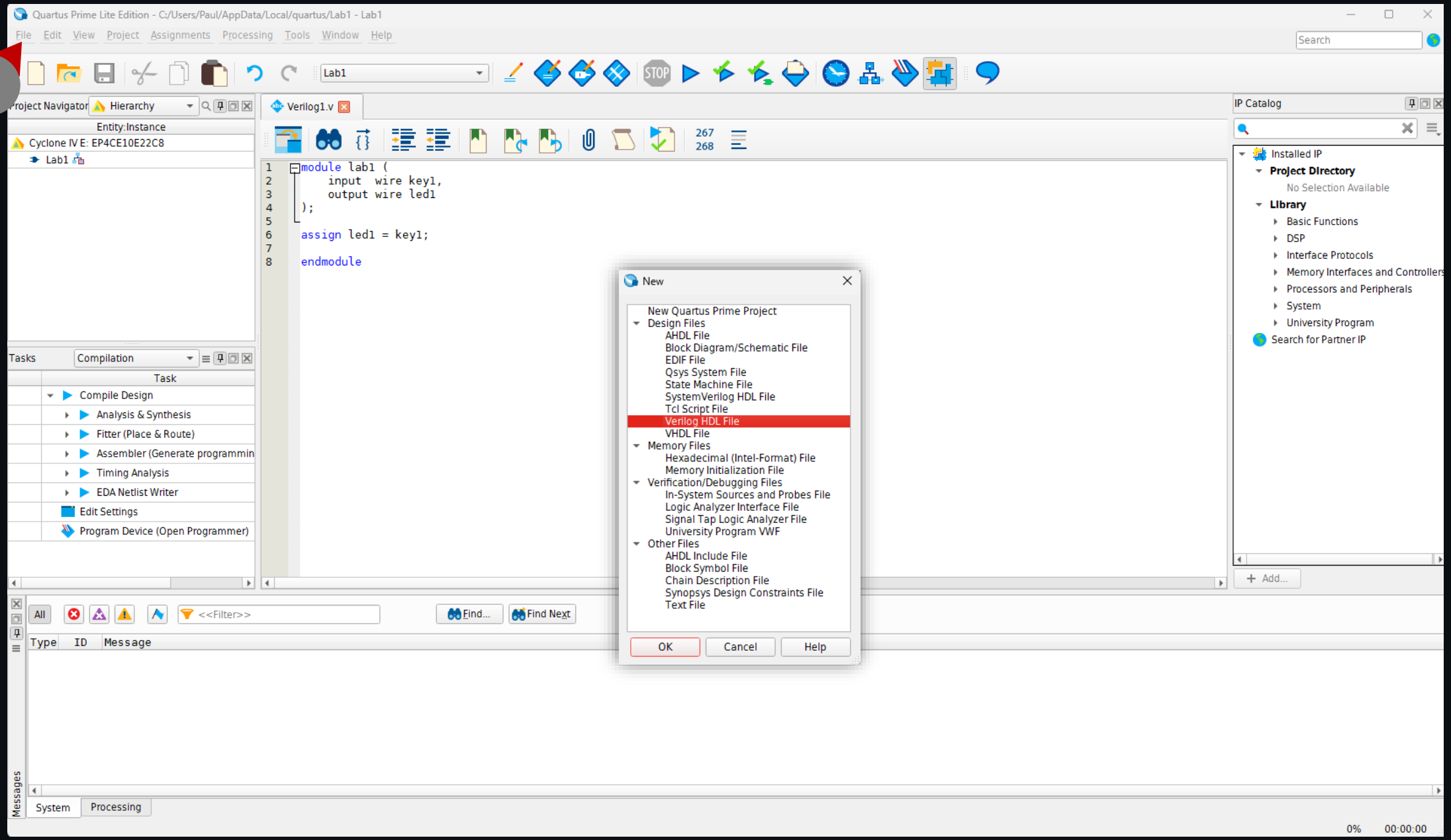

New Project Wizard
Family, Device & Board Settings
Device
Board
Select the family and device you want to target for compilation.
You can install additional device support with the Install Devices command on the Tools menu.
To determine the version of the Quartus Prime software in which your target device is supported, refer to the Device Support List webpage.
Device family
Family: Cyclone IV E
Device: All
Target device
Auto device selected by the Fitter
Specific device selected in 'Available devices' list
Other: n/a
Show in 'Available devices' list
Package: Any
Pin count: Any
Core speed grade: Any
Name filter:
Show advanced devices
Available devices:
Name
Core Voltage
LEs
Total I/Os
GPIOs
Memory Bits
Embedded multiplier 9-bit elements
EP4CE10E22C6 1.2V 10320 92 92 423936 46
EP4CE10E22C7 1.2V 10320 92 92 423936 46
EP4CE10E22C8 1.2V 10320 92 92 423936 46
EP4CE10E22I7 1.2V 10320 92 92 423936 46
Help
< Back
Next >
Finish
Cancel
3
4

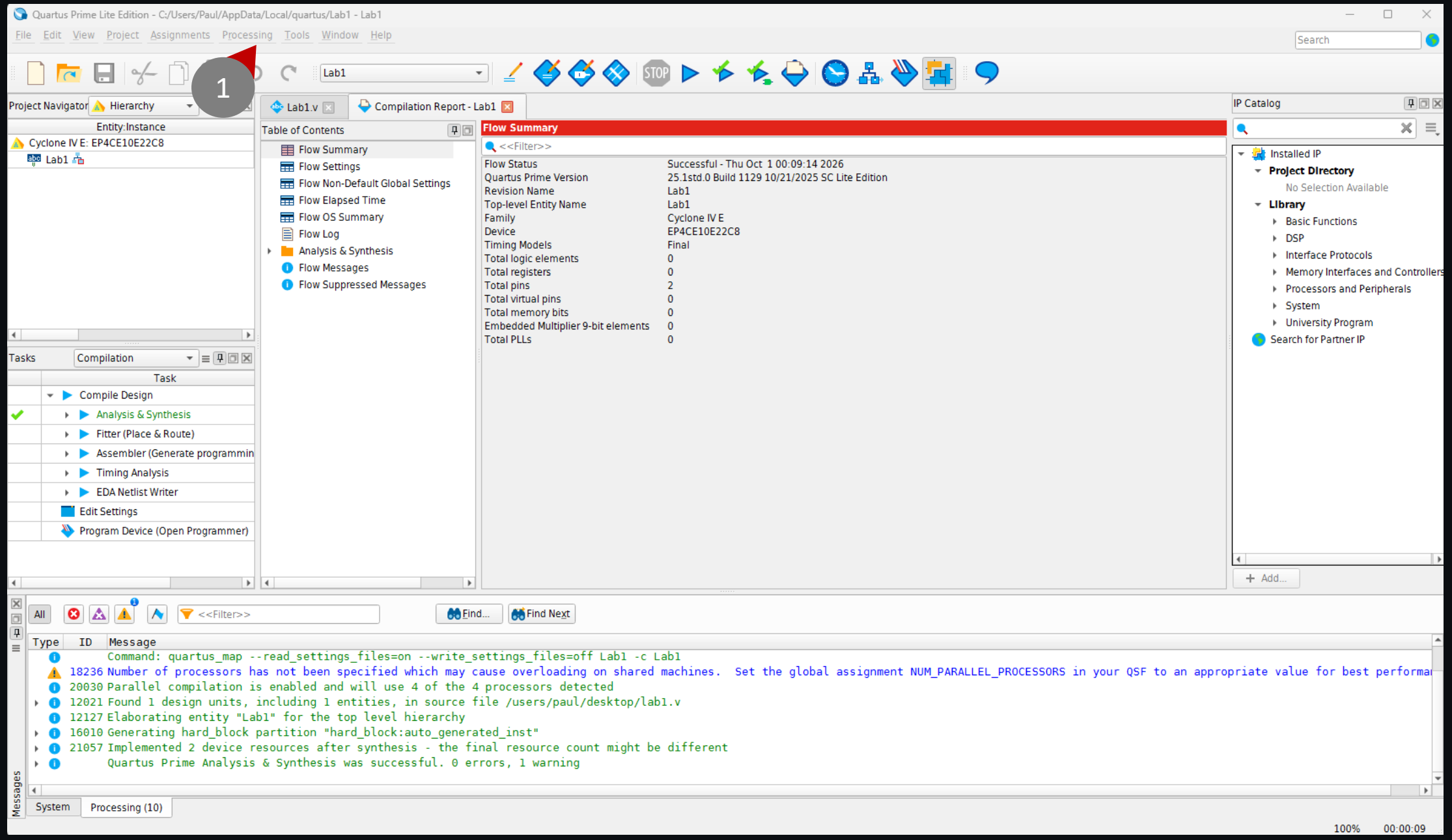

Quartus Prime
Open Quartus Prime projects only from trusted sources. Do you want to continue?
Don't show this message again
Yes
No
5


New Project Wizard
EDA Tool Settings
Specify the other EDA tools used with the Quartus Prime software to develop your project.
EDA tools:
Tool Type
Tool Name
Format(s)
Run Tool Automatically
Design Entry/Synt...
<None>
<None>
Run this tool automatically to synthesize the current design
Simulation
Questa Altera FPGA
Verilog HDL
Run gate-level simulation automatically after compilation
Board-Level
Timing
<None>
Symbol
<None>
Signal Integrity
<None>
Boundary Scan
<None>
Help
< Back
Next >
Finish
Cancel
4

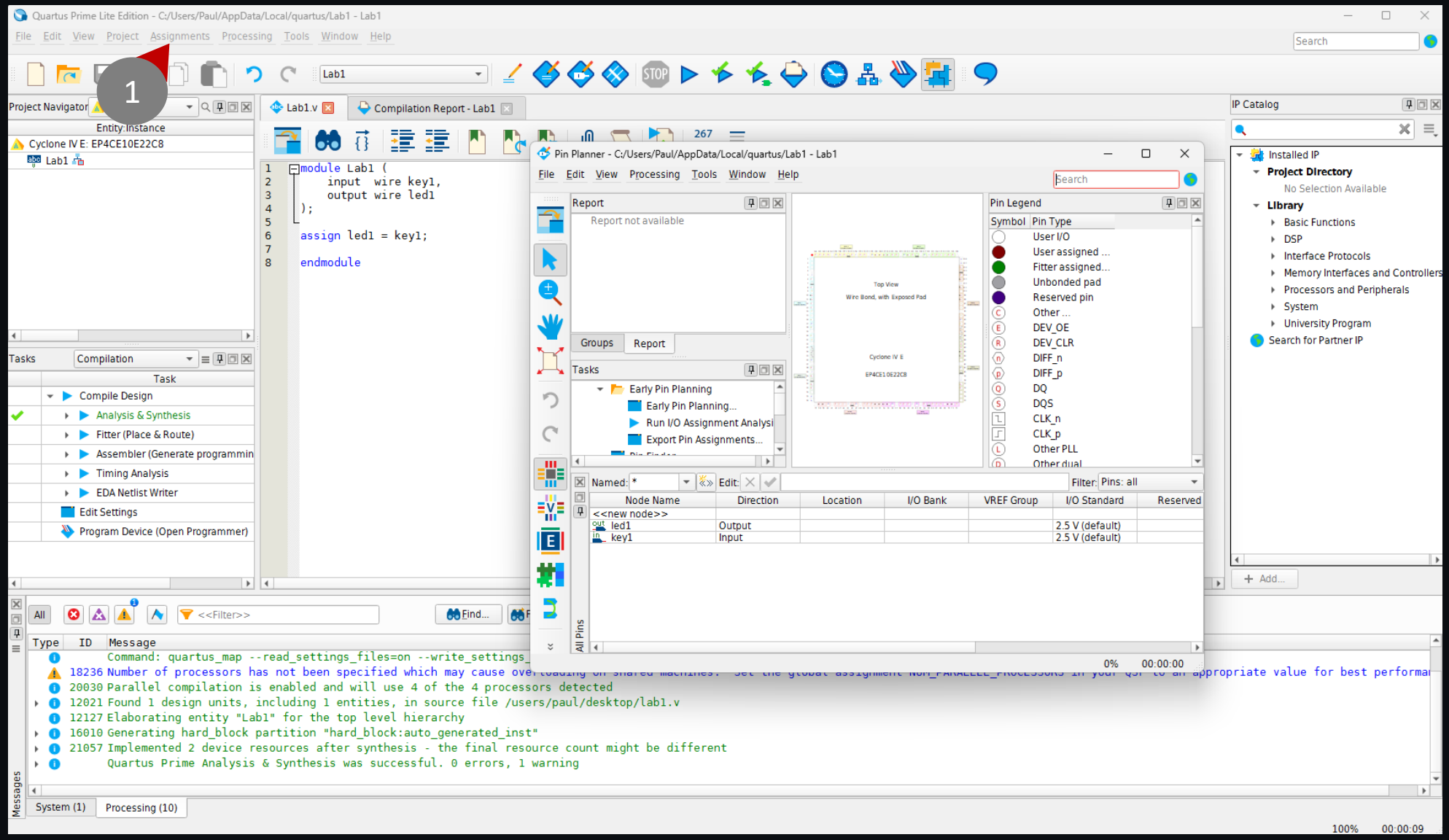
File → Save As → lab1.v

1





Processing → Start → Start Analysis & Synthesis



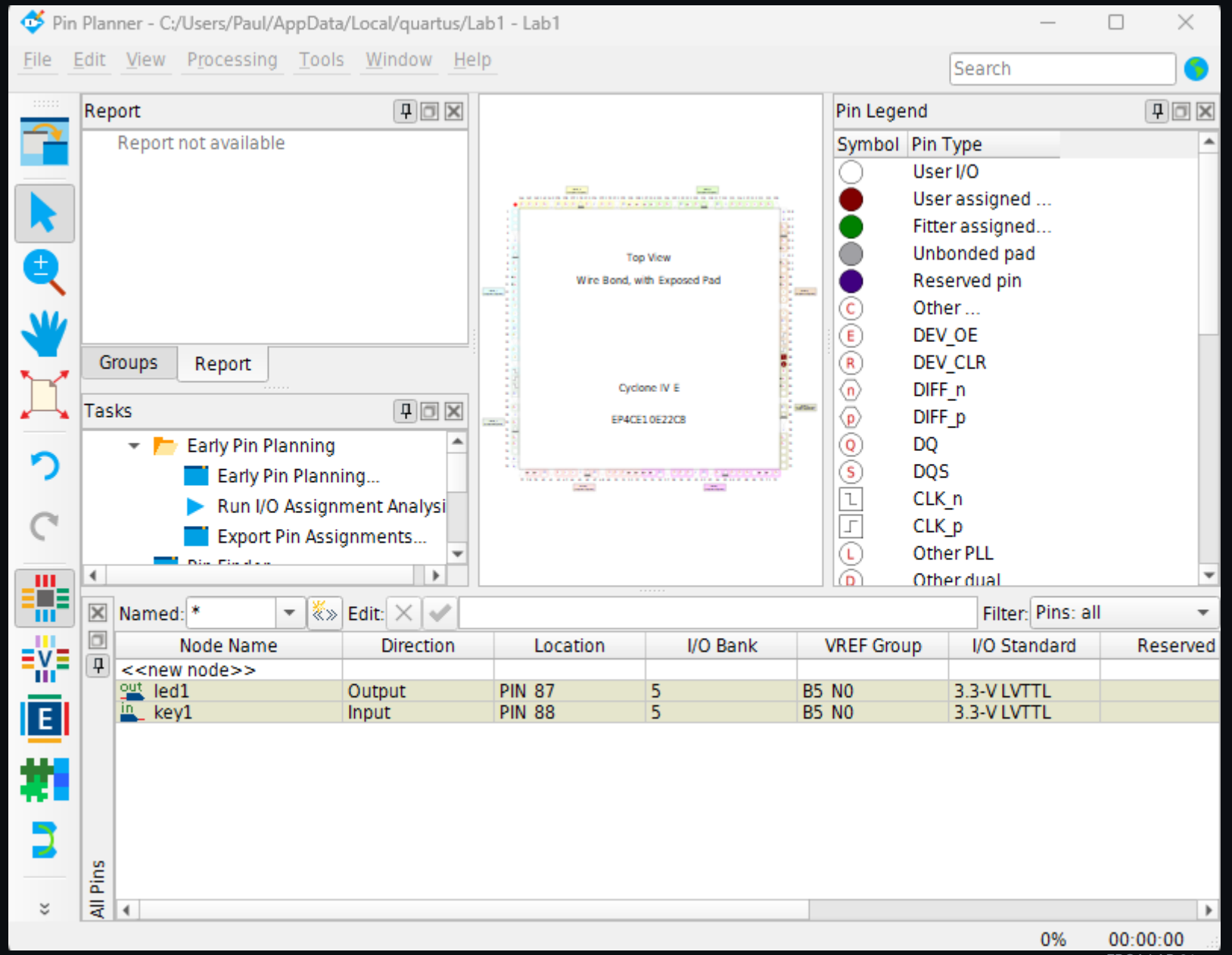
Assignments → Pin Planner

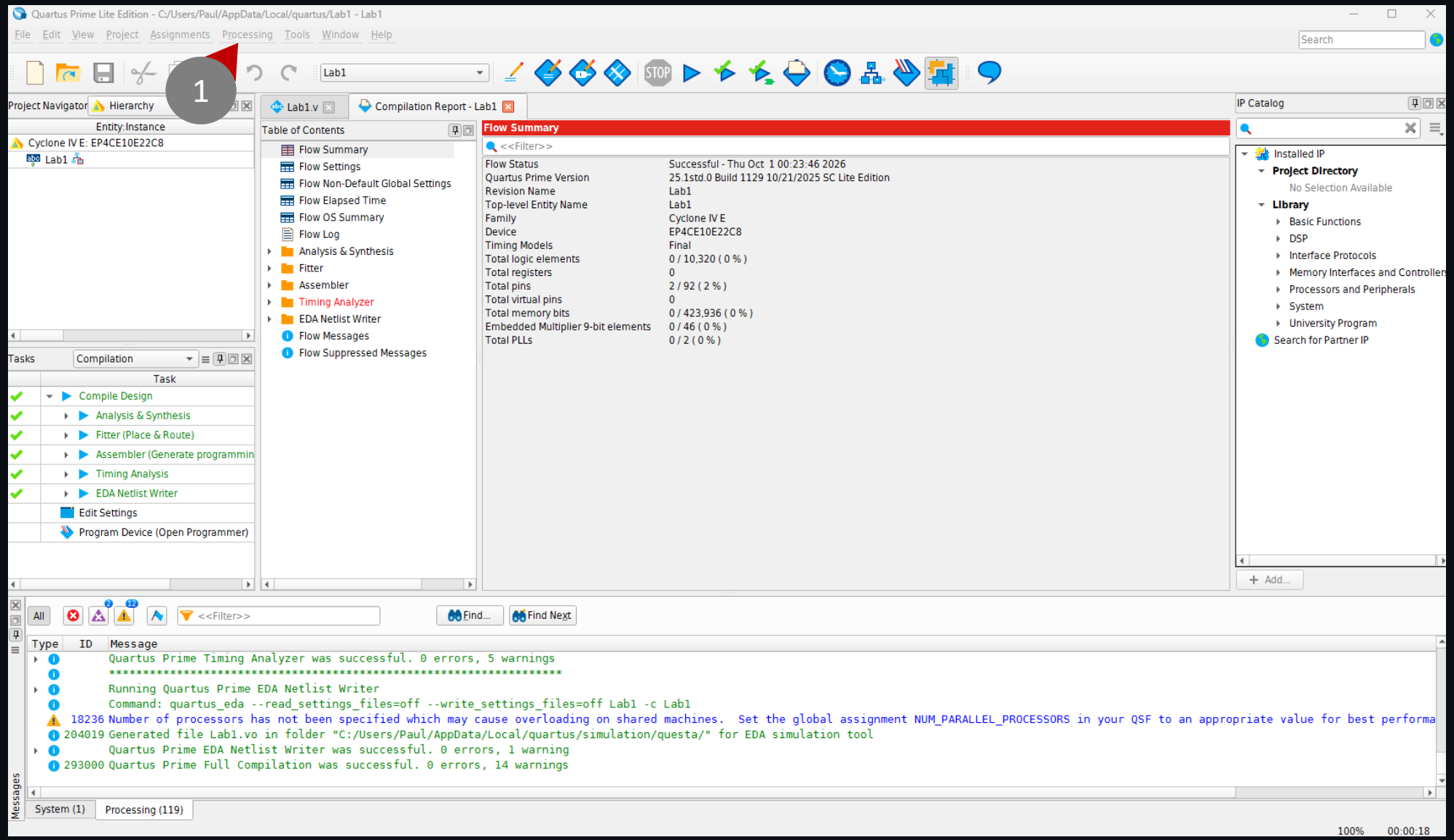
LVTTL = Low-Voltage Transistor-Transistor Logic.

Enter these values in the bottom table:

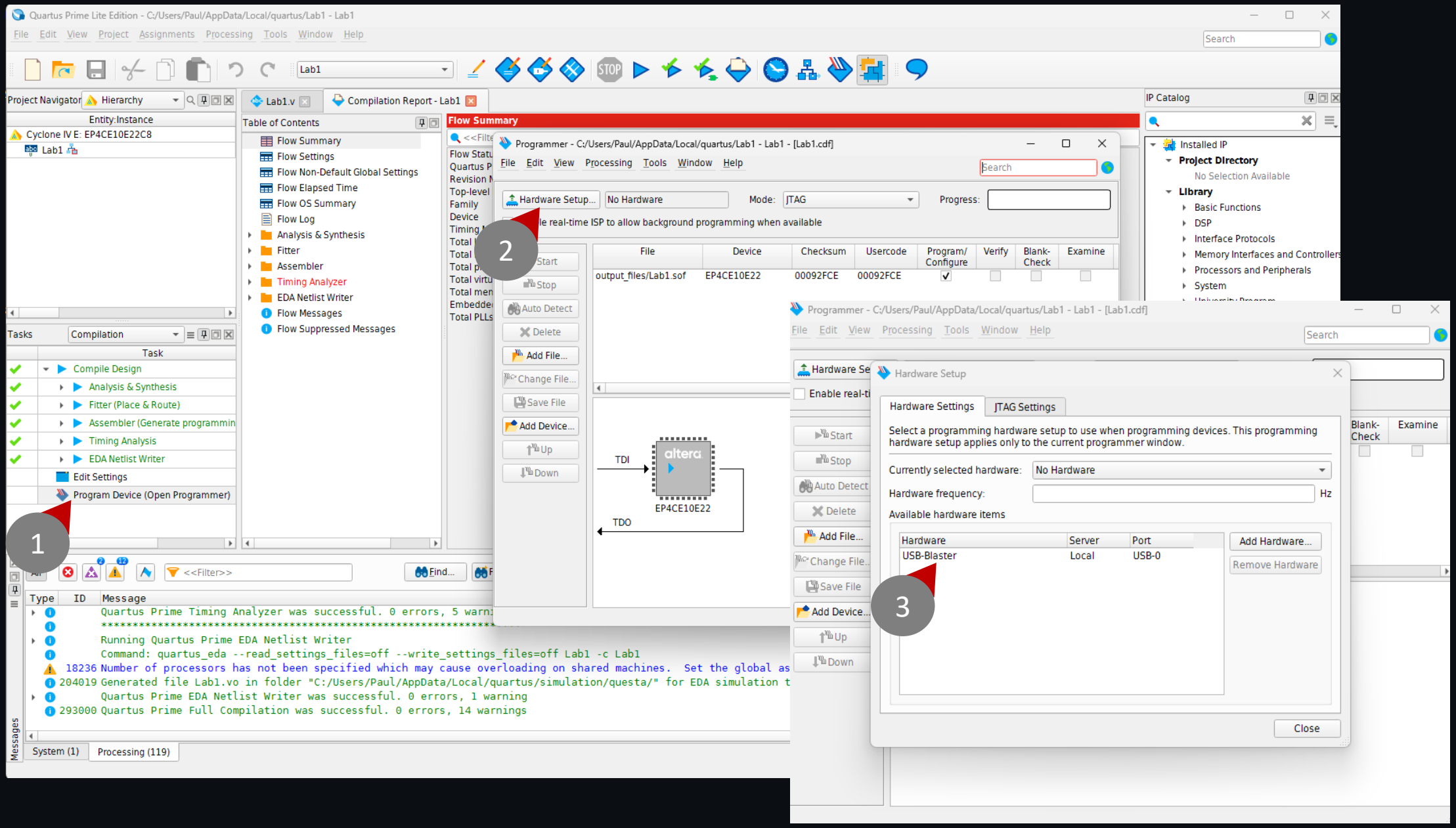
- For `led1`, click the **Location** cell and type `PIN_87`
- For `key1`, click the **Location** cell and type `PIN_88`

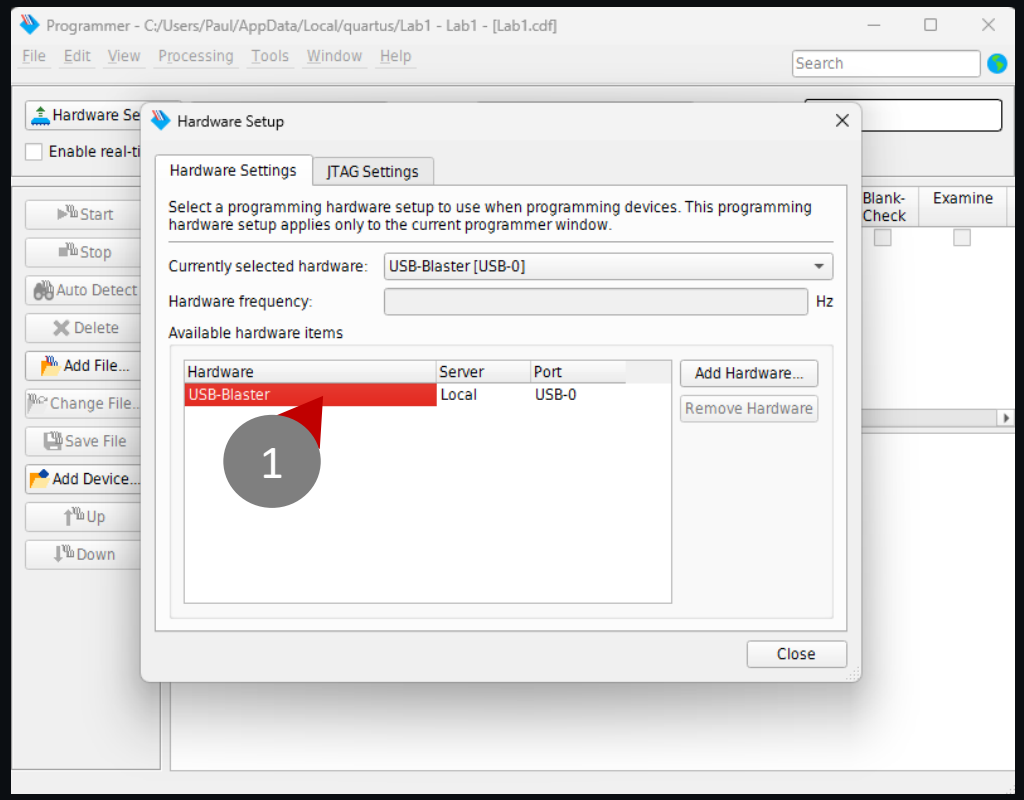
Then change **I/O Standard** for both from `2.5 V (default)` to: `3.3-V LVTTL`

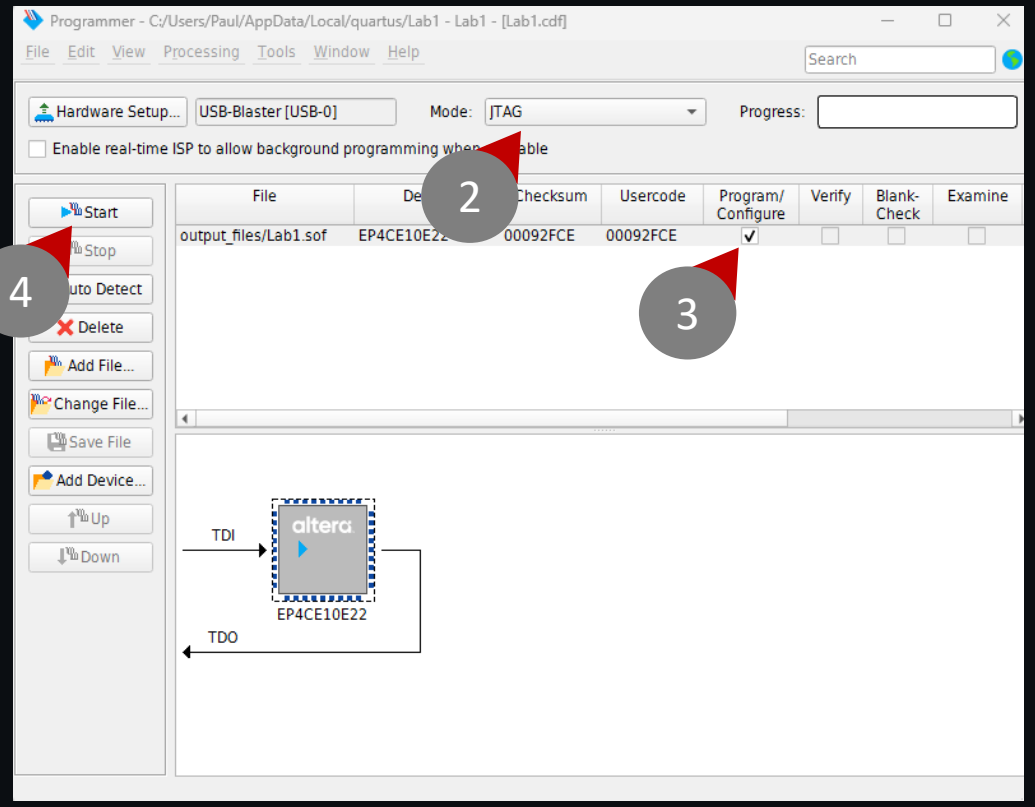


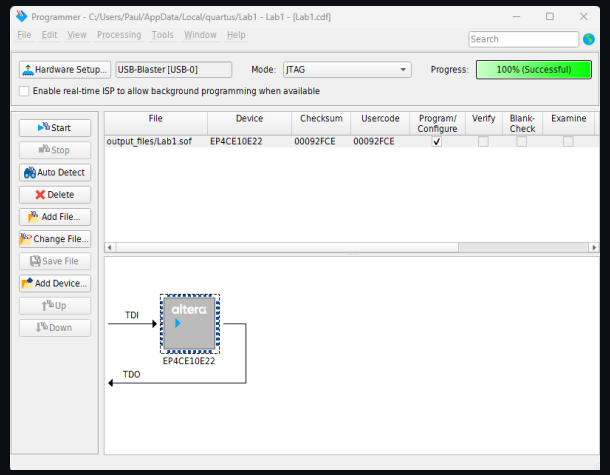


Processing → Start Compilation.


Quartus Prime Lite Edition - C:/Users/Paul/AppData/Local/quartus/Lab1 - Lab1
File Edit View Project Assignments Processing Tools Window Help
Search
Lab1
Project Navigator
Hierarchy
Entity:Instance
Cyclone IV E: EP4CE10E22C8
Lab1
Lab1.v
Compilation Report - Lab1
Table of Contents
Flow Summary
Flow Settings
Flow Non-Default Global Settings
Flow Elapsed Time
Flow OS Summary
Flow Log
Analysis & Synthesis
Fitter
Assembler
Timing Analyzer
EDA Netlist Writer
Flow Messages
Flow Suppressed Messages
Flow Summary
IP Catalog
Installed IP
Project Directory
No Selection Available
Library
Basic Functions
DSP
Interface Protocols
Memory Interfaces and Controllers
Processors and Peripherals
System
Tasks
Compilation
Task
Compile Design
Analysis & Synthesis
Fitter (Place & Route)
Assembler (Generate programmin
Timing Analysis
EDA Netlist Writer
Edit Settings
Program Device (Open Programmer)
1
Programmer - C:/Users/Paul/AppData/Local/quartus/Lab1 - Lab1 - [Lab1.cdf]
File Edit View Processing Tools Window Help
Hardware Setup...
No Hardware
Mode: JTAG
Progress:
2
Start
Stop
Auto Detect
Delete
Add File...
Change File...
Save File
Add Device...
Up
Down
File
Device
Checksum
Usercode
Program/ Configure
Verify
Blank-Check
Examine
output_files/Lab1.sof
EP4CE10E22
00092FCE
00092FCE
TDI
TDO
EP4CE10E22
Hardware Setup
Hardware Settings
JTAG Settings
Select a programming hardware setup to use when programming devices. This programming hardware setup applies only to the current programmer window.
Currently selected hardware:
No Hardware
Hardware frequency:
Hz
Available hardware items
Hardware
Server
Port
USB-Blaster
Local
USB-0
3
Add Hardware...
Remove Hardware
Close
Type ID Message
Quartus Prime Timing Analyzer was successful. 0 errors, 5 warni
Running Quartus Prime EDA Netlist Writer
Command: quartus_eda --read_settings_files=off --write_settings_files=off Lab1 -c Lab1
18236 Number of processors has not been specified which may cause overloading on shared machines. Set the global as
204019 Generated file Lab1.vo in folder "C:/Users/Paul/AppData/Local/quartus/simulation/questa/" for EDA simulation t
Quartus Prime EDA Netlist Writer was successful. 0 errors, 1 warning
293000 Quartus Prime Full Compilation was successful. 0 errors, 14 warnings
Messages
System (1)
Processing (119)


Programmer - C:/Users/Paul/AppData/Local/quartus/Lab1 - Lab1 - [Lab1.cdf]
File Edit View Processing Tools Window Help
Search
Hardware Setup
Hardware Settings
JTAG Settings
Select a programming hardware setup to use when programming devices. This programming hardware setup applies only to the current programmer window.
Currently selected hardware:
USB-Blaster [USB-0]
Hardware frequency:
Hz
Available hardware items
Hardware
Server
Port
USB-Blaster
Local
USB-0
Add Hardware...
Remove Hardware
Close
Start
Stop
Auto Detect
Delete
Add File...
Change File...
Save File
Add Device...
Up
Down
Blank-Check
Examine
1


Programmer - C:/Users/Paul/AppData/Local/quartus/Lab1 - Lab1 - [Lab1.cdf]
File Edit View Processing Tools Window Help
Search
Hardware Setup...
USB-Blaster [USB-0]
Mode:
JTAG
Progress:
Enable real-time ISP to allow background programming when available
Start
Stop
Auto Detect
Delete
Add File...
Change File...
Save File
Add Device...
Up
Down
File
Device
Checksum
Usercode
Program/ Configure
Verify
Blank-Check
Examine
output_files/Lab1.sof
EP4CE10E22
00092FCE
00092FCE
TDI
TDO
altera
EP4CE10E22
2
3
4


Programmer - C:/Users/Paul/AppData/Local/quartus/Lab1 - Lab1 - [Lab1.cdf]
Hardware Setup...
USB-Blaster [USB-0]
Mode:
JTAG
Progress:
100% (Successful)
Enable real-time ISP to allow background programming when available
TDI
TDO
EP4CE10E22

Example: S1 toggles LED1 blinking

FPGA = Field-Programmable Gate Array

Press S1 once → blinking starts. Press S1 again → blinking stops.

Verilog code

```
module Lab1 (
    input  wire clk,
    input  wire key1,
    output wire led1
);

reg [25:0] counter = 0;
reg blink_enable = 0;
reg key_prev = 1;
reg led_state = 0;

always @(posedge clk) begin
    key_prev <= key1;

    // S1 is active-low: pressed = 0
    if (key_prev == 1 && key1 == 0)
        blink_enable <= ~blink_enable;

    if (blink_enable) begin
        if (counter == 25_000_000) begin
            counter <= 0;
            led_state <= ~led_state;
        end else begin
            counter <= counter + 1;
        end
    end else begin
        counter <= 0;
        led_state <= 0;
    end
end

// LED1 is active-low
assign led1 = ~led_state;

endmodule
```

Pin Planner

```
clk   → PIN_23   3.3-V LVTTL
key1 → PIN_88   3.3-V LVTTL
led1 → PIN_87   3.3-V LVTTL
```

What the circuit does

1. The 50 MHz clock drives the circuit.
2. A counter slows 50 MHz down to a visible blink.
3. Each S1 press toggles blink_enable.
4. led_state blinks only when blink_enable = 1.

Run sequence

Save → Start Compilation → Programmer → Start → press S1

Abbreviations

S1 = Switch 1 LED = Light-Emitting Diode
CLK/clk = clock signal I/O = Input/Output
LVTTL = Low-Voltage Transistor-Transistor Logic

S1 toggles LED blinking + buzzer tone

Short Verilog: one button enables two outputs by reusing bits of the same 50 MHz counter.

Verilog source: Lab1.v

```
module Lab1(input clk, input key1,
            output led1, output buzzer);

reg [25:0] cnt = 0;
reg enable = 0, old_key = 1;

always @(posedge clk) begin
    cnt <= cnt + 1;
    old_key <= key1;
    if (old_key & ~key1) enable <= ~enable;
end

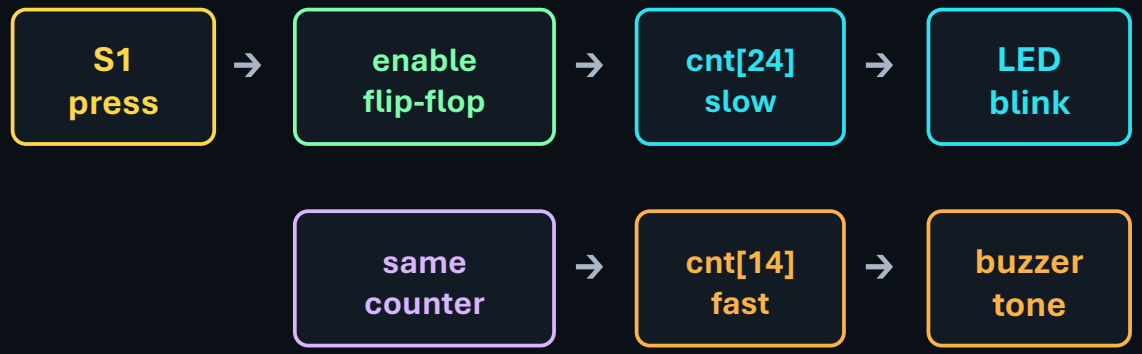
assign led1   = ~(enable & cnt[24]);
assign buzzer =   enable & cnt[14];

endmodule
```

What students observe

Press S1 once → LED blinks and buzzer sounds.
Press S1 again → both stop.
No CPU loop is running: these are parallel hardware signals.

Signal path



Pin Planner

`clk→PIN_23` `key1/S1→PIN_88` `led1→PIN_87` `buzzer→PIN_110`

I/O standard: 3.3-V LVTTL for all four signals

FPGA LAB 01 Core idea: counter bits are hardware signals; they run at the same time.

Abbreviations: S1 = switch/button 1 • LED = Light-Emitting Diode • clk = clock • I/O = Input/Output • LVTTL = Low-Voltage Transistor-Transistor Logic • MHz = megahertz

S1 Counter on the 7-Segment Display

Each press of S1 increments the displayed digit: 0 → 1 → ... → 9 → 0

What this example shows

- **S1 =** push button input
- **clk =** 50 MHz onboard clock
- **n =** decimal counter value (0 to 9)
- **seg =** 7-segment pattern output
- **dig =** digit enable lines
- **Important fix =** the CASE values below repair the digit shapes

Pin reminder:
`clk→PIN_23, key1→PIN_88`
All I/O Standard settings: 3.3-V LVTTL

Short Verilog example

```
module Lab1(input clk, key1,
            output reg [7:0] seg,
            output [3:0] dig);

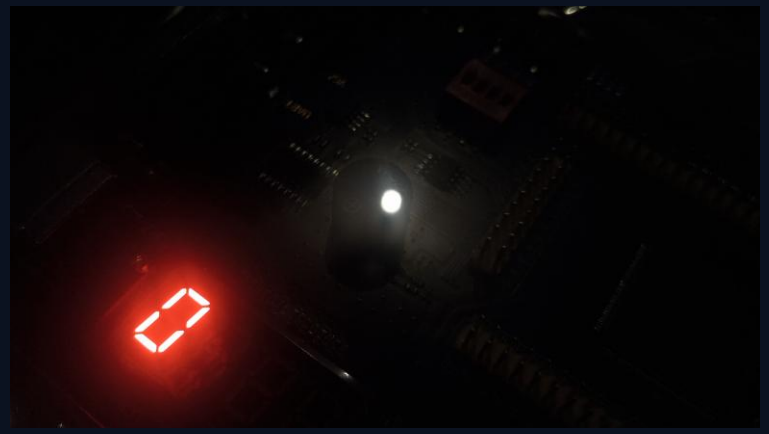
reg [3:0] n = 0;
reg old = 1;

assign dig = 4'b1110;   // one digit ON

always @(posedge clk) begin
    old <= key1;
    if (old & ~key1) n <= (n==9) ? 0 : n+1;
end

always @*
case(n)
  0: seg = 8'hC0;  1: seg = 8'hF9;
  2: seg = 8'hA4;  3: seg = 8'hB0;
  4: seg = 8'h99;  5: seg = 8'h92;
  6: seg = 8'h82;  7: seg = 8'hF8;
  8: seg = 8'h80;  9: seg = 8'h90;
endcase

endmodule
```



Why the display looked strange before: the corrected CASE patterns above match the active-low 7-segment encoding.

8. JTAG and AS — in the first lab we use JTAG

JTAG → .SOF file

- Fast for the lab
- The design goes directly into the FPGA
- The configuration is volatile
- After power is removed, the configuration loaded through JTAG disappears

AS → .POF file

- Writes the serial configuration memory
- At the next power-up, the FPGA can load the design from Flash
- We leave it for a later lab

Today: JTAG + .SOF. We do not enter configuration memory yet.

Abbrev.: JTAG = Joint Test Action Group; AS = Active Serial; SOF = SRAM Object File; POF = Programmer Object File; FPGA = Field-Programmable Gate Array.

9. Check that the PC sees the USB-Blaster

In Windows

1. Connect the USB-Blaster to the PC.

2. Open Device Manager.

3. Look for USB-Blaster / Altera USB-Blaster.

4. If “Unknown device” or a yellow warning sign appears, install the driver from the Quartus package.

In Quartus

Tools → Programmer → Hardware Setup...

Under “Currently selected hardware” you must be able to select USB-Blaster.

If the list is empty, the problem is before the FPGA: USB cable, driver, or programmer.

Do not continue until “USB-Blaster” appears in Hardware Setup.

Abbrev.: PC = Personal Computer; USB = Universal Serial Bus; FPGA = Field-Programmable Gate Array.

10. Create the first Quartus project

Follow the steps exactly, without skipping device selection.

1. **File → New Project Wizard**

   Choose a simple directory, for example C:\FPGA\Lab1.

2. **Project name: Lab1**

   Top-level entity: lab1.

3. **Empty project**

   Do not add other files at this stage.

4. **Family: Cyclone IV E**

   Select the correct family.

5. **Device: EP4CE10E22C8N**

   This is the device on your board.

6. **Finish**

The project is empty, but configured for the correct FPGA.

Abbrev.: FPGA = Field-Programmable Gate Array; EP4CE10E22C8N = exact Cyclone IV E device part number.

11. What does EP4CE10E22C8N mean?

You do not need to memorize it; you must only select it correctly.

EP4CE	10	E22	C	8	N
Cyclone IV E	≈10k logic elements	EQFP package / 144 pins	commercial temperature range	speed grade	lead-free

If we select another FPGA, Quartus may generate an incompatible file or the pin assignment will not match.

For this board: Cyclone IV E → EP4CE10E22C8N

Abbrev.: EQFP = Enhanced Quad Flat Package; EP4CE10E22C8N = device part number; k = thousand.

FPGA Lab 01 — from installation to the first running design

Do not “download Verilog”. Verilog is an HDL language. Install Intel Quartus Prime Lite and the USB-Blaster driver.

1) Install the FPGA tools

Official source: Intel FPGA software downloads
Install: Quartus Prime Lite Edition
Select Cyclone IV support if the installer asks
Install the USB-Blaster driver
Restart Windows if the driver requires it

2) Connect the board

Power: PC USB → USB PWR → POWER button
Programming: PC USB → USB-Blaster → JTAG
Use JTAG, not AS, in Lab 1
The board must be ON before Programmer starts

3) Create the Quartus project

File → New Project Wizard
Project name / top-level entity: lab1
Family: Cyclone IV E
Device: EP4CE10E22C8N

Create a Verilog file named lab1.v:

```
module lab1 (
input  wire key1,
output wire led1
    );

assign led1 = key1;

endmodule
```

4) Assign pins

Assignments → Pin Planner
key1 → PIN_88
led1 → PIN_87
I/O Standard: 3.3-V LVTTL
Then save the project

5) Compile

Processing → Start Compilation
Quartus converts the HDL description into FPGA configuration data
Expected result:
Full Compilation was successful
Output file: output_files/Lab1.sof

6) Program and run

Tools → Programmer
Hardware Setup: USB-Blaster
Mode: JTAG
Add File: Lab1.sof
Tick: Program/Configure
Click: Start
Wait: 100% Successful
The design runs immediately
Test: press S1 → LED1 changes

Abbreviations on this slide

FPGA = Field-Programmable Gate Array; the reconfigurable chip on the board.
HDL = Hardware Description Language; Verilog is an HDL, not a separate program to install.
USB = Universal Serial Bus; USB PWR powers the board, USB-Blaster programs it.
JTAG = Joint Test Action Group; the test/configuration interface used here for programming the FPGA.
AS = Active Serial; flash programming interface, not used in Lab 1. SOF = SRAM Object File; temporary FPGA configuration file. LVTTL = Low-Voltage Transistor-Transistor Logic.

12. Write the simplest possible circuit

A physical button controls a physical LED.

```
module lab1 (
    input  wire key1,
    output wire led1
);

assign led1 = key1;

endmodule
```

What does it describe?

A direct logic connection between the key1 input and the led1 output.

It is not a sequential program

There is no main(), no loop, and no instructions executed one after another.

Saving

File → Save As → lab1.v. Then Project → Add/Remove Files in Project if it was not added automatically.

For the first lab, the code must be simple enough that the main problem remains the hardware workflow.

Abbrev.: LED = Light-Emitting Diode; FPGA = Field-Programmable Gate Array; CPU = Central Processing Unit.

13. Explain the code line by line

`module lab1 (...)`	Defines the circuit / main module.
`input wire key1`	key1 is a signal that enters the FPGA from the button.
`output wire led1`	led1 is a signal that leaves the FPGA toward the LED.
`assign led1 = key1;`	Connects the input to the output logically.
`endmodule`	Ends the module definition.

Important: the FPGA still does not know which physical pin is “key1” and which physical pin is “led1”. That comes next.

Abbrev.: FPGA = Field-Programmable Gate Array; LED = Light-Emitting Diode.

14. Pin assignment - connect Verilog names to the real board

Assignments → Pin Planner

Node Name	Location	I/O Standard
key1	PIN_88	3.3-V LVTTL
led1	PIN_87	3.3-V LVTTL

On the board

S1 / KEY1 → PIN_88
LED1 → PIN_87

These two numbers are the key to the first experiment.

If another board revision does not respond, check the QSF/schematic for that revision before trying random pins.

Abbrev.: I/O = Input/Output; LVTTL = Low-Voltage Transistor-Transistor Logic; QSF = Quartus Settings File; LED = Light-Emitting Diode.

15. Compile - transform Verilog into configurable hardware

1. **Processing → Start Compilation**

   Or use the “Start Compilation” button.

2. **Wait for Analysis & Synthesis → Fitter → Assembler**

   Quartus checks the logic, chooses FPGA resources, and generates the configuration file.

3. **The final message must be “Full Compilation was successful”**

   Warnings may exist; errors must not.

4. **Important file: output_files\Lab1.sof**

   .sof is the file we send through JTAG into the FPGA.

> **If an ERROR appears**
>
> Do not open Programmer yet. First fix the syntax, top-level entity, device, or pin assignment error.

Abbrev.: FPGA = Field-Programmable Gate Array; SOF = SRAM Object File; JTAG = Joint Test Action Group.

16. Open Quartus Programmer

Tools → Programmer

A. Hardware Setup...

Select USB-Blaster.

If it does not appear: check the driver and the programmer USB cable.

B. Mode: JTAG

For the first lab use JTAG.

Do not select Active Serial.

C. The .SOF file

Add File... → select Lab1.sof from output_files.

Check Program/Configure.

Before Start: the board must be powered on and the ribbon cable must be connected to JTAG.

Abbrev.: SOF = SRAM Object File; JTAG = Joint Test Action Group; AS = Active Serial; USB = Universal Serial Bus.

17. Configure the FPGA

1. **Check Hardware: USB-Blaster**

   At the top of the Programmer window.

2. **Check Mode: JTAG**

   Not AS.

3. **Check Program/Configure**

   On the row with the Lab1.sof file.

4. **Press START**

   Progress must reach 100%.

5. **Expected message: 100% (Successful)**

   Now our design is running in the FPGA.

100% - Successful

Abbrev.: FPGA = Field-Programmable Gate Array; JTAG = Joint Test Action Group; AS = Active Serial; SOF = SRAM Object File.

18. Physical test — the moment when the lab “works”



If this happens, the complete chain worked: code → compile → pin assignment → JTAG → FPGA → hardware.

Abbrev.: S1/KEY1 = first push button input; FPGA = Field-Programmable Gate Array; LED = Light-Emitting Diode; JTAG = Joint Test Action Group.

19. Why does “assign led1 = key1” work?

On this board, the button and the LED are active-low.

S1 state	key1	led1	LED
released	1	1	**OFF**
pressed	0	0	**ON**

“Active-low” means that logic value 0 represents the active state.

```
assign led1 = key1;
```

Abbrev.: LED = Light-Emitting Diode; S1 = first push button input.

20. Exercise 2 - modify the project yourselves

Option A: invert the behavior

Change the line to:

assign led1 = ~key1;

Compile → Programmer → Start.

Option B: second button + second LED

key2 → PIN_89
led2 → PIN_86

assign led2 = key2;

The purpose of the exercise is not Verilog itself. The purpose is for you to repeat the complete workflow independently.



Abbrev.: LED = Light-Emitting Diode; PIN = physical FPGA pin.

21. If time remains: the first circuit with CLOCK

The board oscillator is 50 MHz on PIN_23.

```
module blink (
    input  wire clk,
    output wire led1
);

reg [25:0] cnt = 0;
always @(posedge clk)
    cnt <= cnt + 1;

assign led1 = ~cnt[25];

endmodule
```

Pin assignment

clk → PIN_23
led1 → PIN_87

What you observe

The LED blinks without any CPU executing a loop.

Final message

The FPGA is configurable hardware that works in parallel.

Abbrev.: CLOCK = timing signal; MHz = megahertz; CPU = Central Processing Unit; FPGA = Field-Programmable Gate Array.

22. 30-second teaching experiment: power cycle

1. Configure Lab1.sof through JTAG

   The button must control the LED.

2. Turn the board off using POWER

   Power disappears.

3. Turn the board on again

   The design loaded through .SOF/JTAG is not guaranteed to remain in the FPGA.

4. Observe why

   The FPGA is configured at power-up. To make the design persistent, the configuration memory is programmed through AS/.POF — a subject for another lab.

This is the right moment to separate the concepts: “FPGA configuration” and “Flash programming”.

Abbrev.: SOF = SRAM Object File; JTAG = Joint Test Action Group; FPGA = Field-Programmable Gate Array; AS = Active Serial; POF = Programmer Object File.

23. Troubleshooting - what to check when “it does not work”

The board does not power on	USB PWR connected? 5 V source? POWER pressed?
USB-Blaster does not appear	Driver installed? Programmer USB cable OK?
Quartus does not detect the chain	Is the board powered on? Is the ribbon in JTAG, not AS? Is the orientation correct?
Compilation error	Top-level entity = lab1? Is lab1.v in the project? Is the syntax correct?
Programming reaches 100%, but the LED does not respond	Correct pin assignment? key1 PIN_88, led1 PIN_87? Recompiled after changing pins?
The LED works “in reverse”	Return to active-low; test direct assign versus ~key1.

Abbrev.: USB PWR = Universal Serial Bus Power; JTAG = Joint Test Action Group; AS = Active Serial; LED = Light-Emitting Diode.

24. Two-hour plan:

0–15 min	**Board on the bench**	Identify connectors + power rules
15–30 min	**Power-on**	USB PWR, POWER, power check
30–45 min	**USB-Blaster**	JTAG + driver + Hardware Setup
45–70 min	**Quartus**	New Project + device + lab1.v
70–90 min	**Pins + Compile**	PIN_88 / PIN_87 + compile
90–105 min	**Programmer**	JTAG + .SOF + Start
105–115 min	**Test**	S1 → LED1; repeat independently
115–120 min	**Conclusion**	JTAG/.SOF is volatile; what comes in Lab 2

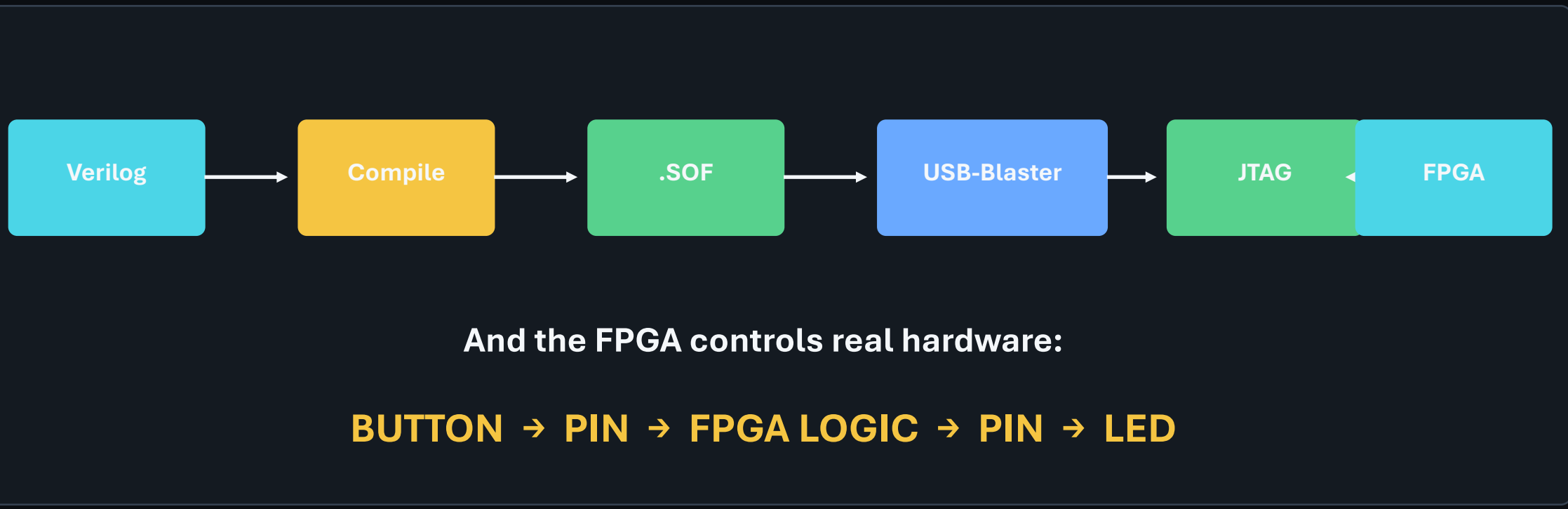
Abbrev.: USB = Universal Serial Bus; PWR = Power; JTAG = Joint Test Action Group; SOF = SRAM Object File; LED = Light-Emitting Diode.

25. Individual checklist - before leaving the lab

- The board is powered correctly at 5 V and can be turned on/off using POWER.
- USB-Blaster is visible in Quartus Hardware Setup.
- The ribbon cable is connected to JTAG, not AS.
- The project uses the EP4CE10E22C8N device.
- lab1.v compiles without errors.
- key1 = PIN_88 and led1 = PIN_87.
- Programmer reaches 100% Successful.
- Pressing S1 changes the LED1 state.
- You can repeat Edit → Compile → Program → Test independently.

Abbrev.: USB = Universal Serial Bus; JTAG = Joint Test Action Group; AS = Active Serial; LED = Light-Emitting Diode.

26. What must remain after the first lab



And the FPGA controls real hardware:

BUTTON → PIN → FPGA LOGIC → PIN → LED

If you can explain and reproduce this chain, Lab 1 has achieved its goal.

Abbrev.: SOF = SRAM Object File; JTAG = Joint Test Action Group; FPGA = Field-Programmable Gate Array; LED = Light-Emitting Diode.

Technical sources / checks for the OMDAZZ board

- OMDAZZ Cyclone IV board manual: 5 V/USB power, power button, JTAG for .SOF, AS for .POF.
- Habr — review of the same OMDAZZ EP4CE10E22C8N family: external USB-Blaster, USB A–B for power, external 50 MHz clock, onboard buttons/LEDs.
- Pin reference for OMDAZZ EP4CE10E22C8N: CLK PIN_23, KEY/SW PIN_88…91, LED PIN_87…84, active-low signals.

Note: for a different PCB revision, the pinout must be confirmed from the schematic/QSF delivered with that revision.

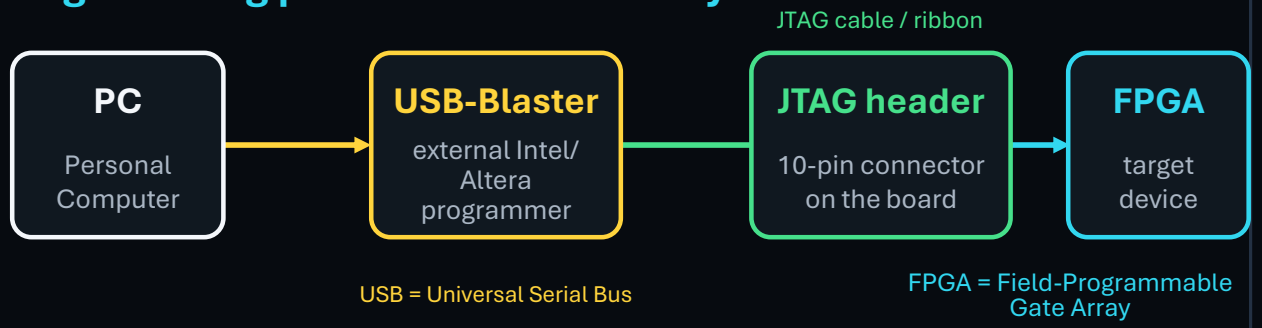
Manual: manuals.plus/ae/1005004865950922
Review: habr.com/ru/articles/749298/
Pin reference: “OMDAZZ EP4CE10E22C8N pin assignments” (board examples / public reference designs)

Abbrev.: USB = Universal Serial Bus; SOF = SRAM Object File; AS = Active Serial; POF = Programmer Object File; CLK = clock; SW = switch; LED = Light-Emitting Diode; QSF = Quartus Settings File; PCB = Printed Circuit Board.

JTAG - the interface used to configure the FPGA

JTAG = Joint Test Action Group. In practice, it means the standardized IEEE 1149.1 hardware interface used for testing, debugging, and device configuration.

Programming path in this laboratory



Important: USB PWR powers the board. USB-Blaster + JTAG configures the FPGA. These are two different USB paths.

Core JTAG signals

- **TCK** — Test Clock — timing signal for the JTAG interface
- **TMS** — Test Mode Select — selects the internal JTAG state
- **TDI** — Test Data In — data from USB-Blaster into the FPGA
- **TDO** — Test Data Out — data from the FPGA back to USB-Blaster

These four lines allow Quartus to detect the device chain and send the configuration bitstream.

For Lab 1

Use JTAG + .SOF

SOF = SRAM Object File. It configures the FPGA temporarily.

Do not use AS yet

AS = Active Serial. It is for serial flash configuration, later.

Power the board first

Quartus detects the JTAG chain only when the board is powered.

Power-off clears it

The .SOF configuration is volatile; it disappears after power-off.

Abbreviations on this slide: JTAG = Joint Test Action Group; IEEE = Institute of Electrical and Electronics Engineers; PC = Personal Computer; USB = Universal Serial Bus; FPGA = Field-Programmable Gate Array; SOF = SRAM Object File; AS = Active Serial; PWR = Power.